```
begin
    # ========= CORE SYMBOLICS =========
    using Symbolics
    using LinearAlgebra

    # ========= DYNAMICS / SOLVER =========
    using DifferentialEquations

    # ========= PLOTTING =========
    using WGLMakie, Makie.Colors   # interactive plots
    WGLMakie.activate!()

    # ========= UTILITIES =========
    using SymbolicUtils
    using BenchmarkTools
    using Random

    # ========= FORMATTING =========
    using PrettyPrint
    import Latexify
    using MacroTools: prettify
    import SymPy
end
```

```
to_hom (generic function with 1 method)
```

```
begin
    @variables t

    Td = Differential(t)

    # scalar time derivative
    TD(x) = expand_derivatives(Td(x))

    # vector/matrix time derivative (broadcast elementwise)
    TDv(X) = expand_derivatives.(Td.(X))

    # aliases
    jac  = Symbolics.jacobian
    hess = Symbolics.hessian

    # homogeneous conversion: [x,y,z] -> [x,y,z,1]
    function to_hom(v)
        length(v) == 4 ? Num.(v) : Num[v[1], v[2], v[3], 1]
    end
end
```

T_rotz (generic function with 1 method)

```
begin
    function eye3()
        Num[
            1 0 0
            0 1 0
            0 0 1
        ]
    end

    function eye4()
        Num[
            1 0 0 0
            0 1 0 0
            0 0 1 0
            0 0 0 1
        ]
    end

    function T_tran(x, y, z)
        Num[
            1 0 0 x
            0 1 0 y
            0 0 1 z
            0 0 0 1
        ]
    end

    T_tran(v) = T_tran(v[1], v[2], v[3])   # multiple dispatch on a vector

    function T_rotx(θ)
        Num[
            1 0       0      0
            0 cos(θ) -sin(θ) 0
            0 sin(θ)  cos(θ) 0
            0 0       0      1
        ]
    end

    function T_roty(θ)
        Num[
             cos(θ) 0 sin(θ) 0
             0      1 0      0
            -sin(θ) 0 cos(θ) 0
             0      0 0      1
        ]
    end

    function T_rotz(θ)
        Num[
            cos(θ) -sin(θ) 0 0
            sin(θ)  cos(θ) 0 0
            0       0      1 0
            0       0      0 1
```

```
        ]
    end
end
```

```
build_pose (generic function with 1 method)
```

```
# ———————————————————— 3. chain = {T1, T2, …, TN} ————————————————————
# A chain is a Vector of 4×4 HTMs:
# chain = [T_tran(q[1], q[2], q[3]), T_rotx(q[4]), T_roty(q[5])]
#
# Pose = T1 * T2 * ... * TN

function build_pose(chain::Vector)
    prod(chain)
end
```

```
make_inertia (generic function with 1 method)
```

```
# ———————————————————— inertia tensor helpers ————————————————————

begin
    # Principal / diagonal inertia tensor
    function inertia_principal(Ix, Iy, Iz)
        Num[
            Ix  0   0
            0   Iy  0
            0   0   Iz
        ]
    end

    # Full symmetric inertia tensor with products of inertia
    function inertia_tensor(Ix, Iy, Iz; Ixy=0, Ixz=0, Iyz=0)
        Num[
            Ix   Ixy  Ixz
            Ixy  Iy   Iyz
            Ixz  Iyz  Iz
        ]
    end

    # Symbolic principal inertia for one body
    function make_I_principal(body_id)
        Ix = Symbolics.variable(Symbol("I_x" * string(body_id)))
        Iy = Symbolics.variable(Symbol("I_y" * string(body_id)))
        Iz = Symbolics.variable(Symbol("I_z" * string(body_id)))

        vars = Num[Ix, Iy, Iz]

        Ibody = inertia_principal(Ix, Iy, Iz)

        return vars, Ibody
    end

    # Symbolic full inertia tensor for one body
    function make_I_tensor(body_id)
        Ix  = Symbolics.variable(Symbol("I_x"  * string(body_id)))
        Iy  = Symbolics.variable(Symbol("I_y"  * string(body_id)))
        Iz  = Symbolics.variable(Symbol("I_z"  * string(body_id)))

        Ixy = Symbolics.variable(Symbol("I_xy" * string(body_id)))
        Ixz = Symbolics.variable(Symbol("I_xz" * string(body_id)))
        Iyz = Symbolics.variable(Symbol("I_yz" * string(body_id)))

        vars = Num[Ix, Iy, Iz, Ixy, Ixz, Iyz]

        Ibody = inertia_tensor(
            Ix, Iy, Iz;
            Ixy = Ixy,
            Ixz = Ixz,
            Iyz = Iyz
        )

```

```
        return vars, Ibody
    end

    # Generic inertia maker
    # mode = :principal → only I_x, I_y, I_z
    # mode = :full      → I_x, I_y, I_z, I_xy, I_xz, I_yz
    function make_inertia(body_id; mode = :full)

        if mode == :principal

            return make_I_principal(body_id)

        elseif mode == :full

            return make_I_tensor(body_id)

        end
    end
end
```

```
multibody_system (generic function with 1 method)
```

```
# ──────────────────────────── 7. bodies / forces / system ────────────────────────────
begin
    # ── force / torque constructors ──────────────────────────────────
    # CONVENTION:
    #   point_local        → always in that body's OWN local frame
    #   force/torque vector → frame keyword decides:
    #       frame = :world → world components (default)
    #       frame = :body  → components in the body's own local frame
    #                        (resolved to world inside compute_Q_total via kin.R)
    #   For frame_force / frame_torque (two bodies):
    #       frame = :world | :body1 | :body2
    point_force(body_id::Int, point_local, force_vector; frame = :world) =
        (; body_id,
           point_local  = Num.(point_local),
           force_vector = Num.(force_vector),
           frame)

    body_torque(body_id::Int, torque_vector; frame = :world) =
        (; body_id,
           torque_vector = Num.(torque_vector),
           frame)

    frame_force(body1_id::Int, point1_local,
                body2_id::Int, point2_local,
                force_vector; frame = :world) =
        (; body1_id,
           point1_local = Num.(point1_local),
           body2_id,
           point2_local = Num.(point2_local),
           force_vector = Num.(force_vector),
           frame)

    frame_torque(body1_id::Int, body2_id::Int, torque_vector; frame = :world) =
        (; body1_id,
           body2_id,
           torque_vector = Num.(torque_vector),
           frame)

    # ── body definition — loads attached to the body ─────────────────
    make_body(id::Int, chain::Vector; forces = [], torques = []) =
        (; id, chain, forces, torques)

    # ── pose access ──────────────────────────────────────────────────
    body_pose(body) = build_pose(body.chain)
    system_poses(sys) = Dict(b.id => body_pose(b) for b in sys.bodies)

    # ── system assembly — loads collected off the bodies ─────────────
    multibody_system(bodies; frame_forces = [], frame_torques = []) =
        (; bodies,
           point_forces = reduce(vcat, [b.forces  for b in bodies]; init = []),
           body_torques = reduce(vcat, [b.torques for b in bodies]; init = []),
           frame_forces,
           frame_torques)
```

```
54 end
```

```
make_params (generic function with 1 method)
```

```julia
# ———————————————— Generic Parameter Block P ————————————————
function make_params(body_ids::Vector{Int};
                     config = Symbol[],
                     config_values = [],
                     stiff_damp = Symbol[],
                     misc = Symbol[],
                     inertia_mode = :full)

    N = length(body_ids)

    # —— configuration parameters ——————————————————————
    config_params = (; (s => Symbolics.variable(s) for s in config)...)
    config_syms = Num[getfield(config_params, s) for s in config]
    config_value_dict = Dict(config_syms[i] => config_values[i]
                    ⇥ ⇥  for i in eachindex(config_values))

    # —— inertial parameters, auto-generated per body ——————
    M_body = Num[Symbolics.variable(Symbol("m_" * string(id))) for id in body_ids]

    I_body = Vector{Matrix{Num}}(undef, N)
    I_vars = Vector{Vector{Num}}(undef, N)
    for (j, id) in enumerate(body_ids)
        vars, Ib = make_inertia(id; mode = inertia_mode)
        I_vars[j] = vars
        I_body[j] = Ib
    end

    com = [Num[Symbolics.variable(Symbol("c_x" * string(id))),
               Symbolics.variable(Symbol("c_y" * string(id))),
               Symbolics.variable(Symbol("c_z" * string(id)))]
           for id in body_ids]

    # —— stiffness / damping ———————————————————————————
    stiff_damp_params = (; (s => Symbolics.variable(s) for s in stiff_damp)...)
    stiff_damp_syms = Num[getfield(stiff_damp_params, s) for s in stiff_damp]

    # —— miscellaneous ——————————————————————————————————
    misc_params = (; (s => Symbolics.variable(s) for s in misc)...)
    misc_syms = Num[getfield(misc_params, s) for s in misc]

    # —— main parameter structure (no g — gravity is a force input) —
    P = (; N, body_ids,
           config = config_params, config_values, config_value_dict,
           M_body, I_body, com,
           stiff_damp = stiff_damp_params,
           misc = misc_params)

    # —— flat symbolic parameter vector ————————————————
    # order: config | body-wise (m, I vars, com) | stiff_damp | misc
    p_syms = Num[]
    append!(p_syms, config_syms)

    inertial_syms = Num[]
```

```
    for j in 1:N
        push!(inertial_syms, M_body[j])
        append!(inertial_syms, I_vars[j])
        append!(inertial_syms, com[j])
    end
    append!(p_syms, inertial_syms)

    append!(p_syms, stiff_damp_syms)
    append!(p_syms, misc_syms)

    # ── grouped view for building numeric values / debugging ─────
    param_inputs = (; config = config_syms,
                      inertial = inertial_syms,
                      stiff_damp = stiff_damp_syms,
                      misc = misc_syms)

    return P, p_syms, param_inputs
end
```

```
Omega (generic function with 1 method)
```

```
begin
    # inverse of skew: pulls ω out of a skew-symmetric matrix
    function vee_skew(S)
        Num[
            S[3, 2]
            S[1, 3]
            S[2, 1]
        ]
    end

    # world-frame angular velocity of a body given its 4x4 pose T
    function Omega(T)
        R    = T[1:3, 1:3]
        Rdot = TDv(R)
        Ω    = (Rdot * transpose(R))   # Ṙ Rᵀ = skew(ω_world)
        ω    = (vee_skew(Ω))
        return ω
    end
end
```

build_kinematics (generic function with 1 method)

```
function build_kinematics(sys, q, P)
    B = length(sys.bodies)
    T_body_list = Vector{Matrix{Num}}(undef, B)
    R_list      = Vector{Matrix{Num}}(undef, B)
    rG_list     = Vector{Vector{Num}}(undef, B)
    vG_list     = Vector{Vector{Num}}(undef, B)
    ω_list      = Vector{Vector{Num}}(undef, B)
    Iglob_list  = Vector{Matrix{Num}}(undef, B)
    for i in 1:B
        body = sys.bodies[i]
        # Body pose from HTM chain
        T_body = build_pose(body.chain)
        T_body_list[i] = T_body
        # Rotation matrix of body
        R = T_body[1:3, 1:3]
        R_list[i] = R
        # COM position in world frame (P.com[i] is body-local)
        rG = (T_body * to_hom(P.com[i]))[1:3]
        rG_list[i] = rG
        # COM velocity in world frame
        vG = TDv(rG)
        vG_list[i] = vG
        # Angular velocity in world frame
        ω_list[i] = Omega(T_body)
        # Inertia tensor expressed in world frame
        Iglob_list[i] = R * P.I_body[i] * transpose(R)
    end
    return (;
        T_body = T_body_list,
        R      = R_list,
        rG     = rG_list,
        vG     = vG_list,
        omega  = ω_list,
        Iglob  = Iglob_list
    )
end
```

```
compute_KE (generic function with 1 method)
```

```
begin
    # ═══════════════ 8. KINETIC ENERGY (König) ═══════════════
    #
    # For each rigid body:
    #
    #   KE_i = 1/2 m_i vG_iᵀ vG_i  +  1/2 ω_iᵀ I_world_i ω_i
    #
    # m_i from P.M_body (auto-generated symbols)
    # vG, ω, Iglob from build_kinematics — all world frame, consistent pair
    #
    function KE_body(m, I_global, vG, ω)
        KE_tr  = (1//2) * m * dot(vG, vG)
        KE_rot = (1//2) * dot(ω, I_global * ω)
        return KE_tr + KE_rot, KE_tr, KE_rot
    end

    function compute_KE(sys, P, kin)
        N = length(sys.bodies)
        KE_each     = Vector{Num}(undef, N)
        KE_tr_each  = Vector{Num}(undef, N)
        KE_rot_each = Vector{Num}(undef, N)
        KE_total = Num(0)
        for i in 1:N
            KE_i, KE_tr, KE_rot = KE_body(
                P.M_body[i],
                kin.Iglob[i],
                kin.vG[i],
                kin.omega[i]
            )
            KE_each[i]     = KE_i
            KE_tr_each[i]  = KE_tr
            KE_rot_each[i] = KE_rot
            KE_total += KE_i
        end
        return (;
            KE_total,
            KE_each,
            KE_trans = KE_tr_each,
            KE_rot   = KE_rot_each
        )
    end
end
```

```
compute_Q_total (generic function with 1 method)
```

```
begin
    # ═══════════════ 9. GENERALIZED FORCES (virtual work) ═══════════════
    #
    # Q_total = Σ point forces + Σ body torques
    #         + Σ frame forces + Σ frame torques
    #
    # All computation in world frame; :body vectors rotated to world

    function compute_Q_total(sys, q, kin)

        n    = length(q)
        qdot = TD.(q)

        Q_total = zeros(Num, n)

        # ── 1. POINT FORCES:  Q = Jᵀ F,  J = ∂r_point/∂q ──────────
        for f in sys.point_forces
            T_body  = kin.T_body[f.body_id]
            r_point = (T_body * to_hom(f.point_local))[1:3]
            F_world = f.frame == :body  ? kin.R[f.body_id] * f.force_vector :
                      f.force_vector
            J_point = jac(r_point, q)
            Q_total .+= vec(transpose(J_point) * F_world)
        end

        # ── 2. BODY TORQUES:  Q = Jωᵀ τ,  Jω = ∂ω/∂q̇ ─────────────
        for τ in sys.body_torques
            ω_body  = kin.omega[τ.body_id]
            τ_world = τ.frame == :body  ? kin.R[τ.body_id] * τ.torque_vector :
                      τ.torque_vector
            Jω = jac(ω_body, qdot)
            Q_total .+= vec(transpose(Jω) * τ_world)
        end

        # ── 3. FRAME FORCES (action-reaction between two points) ──
        #    F on body2 at p2, -F on body1 at p1, via r_rel = p2 - p1
        for ff in sys.frame_forces
            p1 = (kin.T_body[ff.body1_id] * to_hom(ff.point1_local))[1:3]
            p2 = (kin.T_body[ff.body2_id] * to_hom(ff.point2_local))[1:3]
            F_world = ff.frame == :body1 ? kin.R[ff.body1_id] * ff.force_vector :
                      ff.frame == :body2 ? kin.R[ff.body2_id] * ff.force_vector :
                      ff.force_vector
            J_rel = jac(p2 .- p1, q)
            Q_total .+= vec(transpose(J_rel) * F_world)
        end

        # ── 4. FRAME TORQUES (relative couple between two bodies) ─
        #    τ on body2, -τ on body1, via ω_rel = ω2 - ω1
        for ft in sys.frame_torques
            ω_rel   = kin.omega[ft.body2_id] .- kin.omega[ft.body1_id]
            τ_world = ft.frame == :body1 ? kin.R[ft.body1_id] * ft.torque_vector :
                      ft.frame == :body2 ? kin.R[ft.body2_id] * ft.torque_vector :
                      ft.frame == :world ? ft.torque_vector :
```

```
                    →| →| →| →| →|    ft.torque_vector
            Jω_rel = jac(ω_rel, qdot)
            Q_total .+= vec(transpose(Jω_rel) * τ_world)
        end

        return Q_total
    end
end
```

```
compute_M_h (generic function with 1 method)
```

```
begin
    # ═══════════════ 10. MASS MATRIX AND h VECTOR ═══════════════
    #
    # Euler-Lagrange kinetic part:
    #
    #   EL = d/dt(∂T/∂q̇) - ∂T/∂q  =  M(q) q̈ + h(q, q̇)
    #
    # Gravity and applied loads are NOT here — they live in Q_total.
    # Final dynamics:  M q̈ + h = Q   ⇒   q̈ = M⁻¹(Q - h)
    function compute_M_h(KE_total, q; method = :hessian)
        qdot  = TD.(q)
        qddot = TD.(qdot)

        # ── Euler-Lagrange kinetic part ──────────────────────────
        dKE_dqdot = vec(jac([KE_total], qdot))     # ∂T/∂q̇
        dKE_dq    = vec(jac([KE_total], q))        # ∂T/∂q
        EL = TDv(dKE_dqdot) .- dKE_dq

        # ── mass matrix ──────────────────────────────────────────
        if method == :hessian
            # T exactly quadratic in q̇ ⇒ Hessian is M (symmetric)
            M = hess(KE_total, qdot)
        elseif method == :accelcoeff
            # EL linear in q̈ ⇒ coefficient matrix is M
            M = jac(EL, qddot)
        end

        # ── h by algebraic cancellation (robust, self-checking) ──
        # EL = M q̈ + h identically ⇒ h = EL - M q̈ ; the q̈ terms
        # cancel by expansion. If any q̈ survives in h, extraction
        # of M was wrong — visible on inspection, never silent.
        # ── h: substitute q̈ = 0 into EL (no expansion — memory-safe) ──
        zero_acc = Dict(qddot .=> 0)
        h_vec = substitute.(EL, Ref(zero_acc))

        return M, h_vec
    end
end
```

```
build_symbolic_model (generic function with 1 method)
```

```
    begin
        # ═══════════════ 11. THE SYMBOLIC MODEL ═══════════════
        struct SymbolicModel
            sys       # multibody system (bodies + loads)
            P         # parameter block (M_body, I_body, com, config…)
            q         # generalized coordinates
            qdot      # generalized velocities
            x         # state vector [q; qdot]
            p_syms    # ordered symbolic parameter vector (build_function order)
            kin       # kinematics table
            KE        # kinetic energy data
            Q         # total generalized force vector
            M         # mass matrix
            h         # nonlinear vector
        end

        function build_symbolic_model(sys, q, P, p_syms; method = :hessian)
            qdot = TD.(q)
            x    = vcat(q, qdot)
            kin = build_kinematics(sys, q, P)
            KE_data = compute_KE(sys, P, kin)
            Q_total = compute_Q_total(sys, q, kin)
            M, h_vec = compute_M_h(KE_data.KE_total, q; method = method)
            return SymbolicModel(sys, P, q, qdot, x, p_syms,
                        ↦ ↦  kin, KE_data, Q_total, M, h_vec)
        end
    end
```

```
run_simulation (generic function with 1 method)
```

```julia
begin
    # ═══════════════ 12. COMPILE + SIMULATE ═══════════════
    #
    # statify (global, boundary cell): ψ(t), D(ψ(t)) → q_dummy[i]
    # build_function: THE symbolic → numeric boundary
    #
    #   u = [q; qdot]   – ordering fixed by states_dyn in the boundary cell
    #   p = numeric values in the order of model.p_syms  (= p_vals)

    function compile_ode(model)
        n = length(model.q)

        # ── symbolic → plain-variable form ──────────────────────
        M_s = statify.(model.M)
        h_s = statify.(model.h)
        Q_s = statify.(model.Q)

        # ── compile to numeric functions ────────────────────────
        M_fun =build_function(M_s, states_dync, model.p_syms; expression = Val(false),
        cse = true)[1]
        h_fun =build_function(h_s, states_dync, model.p_syms; expression = Val(false),
        cse = true)[1]
        Q_fun =build_function(Q_s, states_dync, model.p_syms; expression = Val(false),
        cse = true)[1]

        # ── ODE right-hand side:  M q̈ = Q - h ──────────────────
        function dynamics!(du, u, p, tnow)
            M_now = M_fun(u, p)
            rhs   = vec(Q_fun(u, p)) .- vec(h_fun(u, p))
            du[1:n]    .= u[n+1:2n]
            du[n+1:2n] .= M_now \ rhs
            return nothing
        end
        return dynamics!
    end

    function run_simulation(model, u0, tspan;
                    p, solver = Tsit5(),
                    reltol = 1e-8, abstol = 1e-8)
        prob = ODEProblem(compile_ode(model), u0, tspan, p)
        return solve(prob, solver; reltol = reltol, abstol = abstol)
    end
end
```

```
sympy_simplify (generic function with 1 method)
```

```
begin
    # ═══════ sympy_simplify — Symbolics → SymPy → simplify → Symbolics ═══════
    #
    # Input must already be STATIFIED (plain symbols only — no ψ(t), no D(ψ(t))).
    # That's exactly what sympylify guarantees before calling this.
    #
    # Round-trip strategy (no string parsing either way):
    #   1. collect the expression's variables
    #   2. compile it with build_function, CALL it on SymPy symbols
    #      → the same expression, now living in SymPy
    #   3. SymPy.simplify
    #   4. lambdify back to a Julia function, CALL it on the original
    #      Symbolics variables → simplified expression, back in Symbolics
    #
    function sympy_simplify(expr)
        vars = Symbolics.get_variables(expr)
        isempty(vars) && return expr                        # pure constant
        k = length(vars)
        # fresh, identifier-safe SymPy names (q_dummy[1] etc. are not valid identifiers)
        spsyms = [SymPy.symbols("v$i") for i in 1:k]
        f  = Symbolics.build_function(expr, vars...; expression = Val(false))
        sp = SymPy.simplify(f(spsyms...))                   # expression now in SymPy, simplified
        back = SymPy.lambdify(sp, spsyms)                   # SymPy → Julia function
        return back(Num.(vars)...)                          # evaluate on Symbolics vars → Num
    end
end
```

$$[\mathbf{xh}(t),\ \mathbf{zh}(t),\ \boldsymbol{\theta}\mathbf{h}(t),\ \boldsymbol{\psi}\mathbf{h}(t)]$$

```
begin
    # ═══════ REPLACED CELL — 4-DOF AUV generalized coordinates ═══════
    #
    #   q = [ xh   zh   θh   ψh ]  =  [ surge pos, heave pos, pitch, yaw ]
    #
    # Sway and roll are dropped: torpedo hull is metacentrically stable in
    # roll (CB above CG) and has no sway actuation. World frame: z UP.
    @variables xh(t) zh(t) θh(t) ψh(t)
    q = Num[xh, zh, θh, ψh]
end
```

chains =

$$\left[\left[\begin{bmatrix} 1 & 0 & 0 & \mathbf{xh}(t) \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \mathbf{zh}(t) \\ 0 & 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} \cos(\psi\mathbf{h}(t)) & -\sin(\psi\mathbf{h}(t)) & 0 & 0 \\ \sin(\psi\mathbf{h}(t)) & \cos(\psi\mathbf{h}(t)) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} \cos(\theta\mathbf{h}(t)) & 0 & \sin(\theta\mathbf{h}(t)) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin(\theta\mathbf{h}(t)) & 0 & \cos(\theta\mathbf{h}(t)) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}\right]\right]$$

```
# ═══════ REPLACED CELL — one body, one chain (was 6 chains) ═══════
#
#   hull pose:  translate (x, 0, z) → yaw ψ about world z → pitch θ about hull y
#
chains = [
    [T_tran(q[1], 0, q[2]), T_rotz(q[4]), T_roty(q[3])]
]
```

0.09

```
begin
    # ═══════ REPLACED CELL — hull geometry (plain numbers only) ═══════
    # ⚠ PLACEHOLDER values sized for the CAD torpedo (~0.9 m, Ø0.15 m).
    #   Measure the real ones from SolidWorks mass properties and replace.
    m_hull_val = 14.0                       # kg
    W_val      = m_hull_val * 9.81          # 137.34 N
    com_val    = [0.0, 0.0, -0.02]          # CG 2 cm BELOW hull origin (body frame, z
    up)
    cb_val     = [0.0, 0.0,  0.01]          # CB 1 cm ABOVE origin → pitch restoring

    r_thr      = [-0.48, 0.0, 0.0]          # thruster on axis, at the tail
    x_fin      = -0.42                      # fin axial station
    R_fin      = 0.09                       # fin mount radius off the hull axis
end
```

$$[([0,\ \mathrm{Mq_}\,\frac{\mathrm{d}\,\theta\mathrm{h}(t)}{\mathrm{d}t} + \mathrm{Mqq_}\,\frac{\mathrm{d}\,\theta\mathrm{h}(t)}{\mathrm{d}t}\sqrt{\mathrm{veps}^2 + \left(\frac{\mathrm{d}\,\theta\mathrm{h}(t)}{\mathrm{d}t}\right)^2},\ 0],\ 1,\ :\mathrm{body}),\ ([0,\ 0,\ \mathrm{Nr_}\,\frac{\mathrm{d}\,\psi\mathrm{h}(t)}{\mathrm{d}t} + \mathrm{Nrr_}$$

```
begin
    # ═══════ REPLACED CELL — actuator + hydrodynamic force inputs ═══════
    #
    # cfg(s) creates the IDENTICAL symbol make_params regenerates from
    # config_names later — same trick that made Vt/kd sweepable through p.
    cfg(s) = Symbolics.variable(s)

    T_thr_s = cfg(:T_thr)                          # thruster force [N]
    δ1_s = cfg(:δ1); δ2_s = cfg(:δ2); δ3_s = cfg(:δ3)   # the 3 servo fins [rad]
    ρw = cfg(:ρw); Bb = cfg(:Bb); veps = cfg(:veps)
    Xu_ = cfg(:Xu_); Xuu_ = cfg(:Xuu_); Zw_ = cfg(:Zw_); Zww_ = cfg(:Zww_)
    Mq_ = cfg(:Mq_); Mqq_ = cfg(:Mqq_); Nr_ = cfg(:Nr_); Nrr_ = cfg(:Nrr_)
    Afin = cfg(:Afin); CLa = cfg(:CLa)

    # ── hull kinematics for force construction (pure chain algebra) ──
    T_hull = build_pose(chains[1])
    R_hull = T_hull[1:3, 1:3]

    # body-frame velocity of any hull-local point
    vbody(pl) = R_hull' * TDv((T_hull * to_hom(pl))[1:3])

    νo   = vbody([0, 0, 0])                # hull-origin velocity, body frame
    u_b  = νo[1]                           # surge
    w_b  = νo[3]                           # heave
    qp_b = TD(q[3])                        # pitch rate
    r_b  = TD(q[4])                        # yaw rate

    # ── THE regularization — verbatim from the 9-DOF tyre model ──
    inv_u   = u_b / (u_b^2 + veps^2)       # smooth 1/u, finite at rest
    sabs(v) = v * sqrt(v^2 + veps^2)       # smooth v·|v| for quadratic drag
    qdyn    = (1//2) * ρw * (u_b^2 + veps^2)   # fin dynamic pressure

    # ── 3 servo fins at 120° around the tail (Y-tail, from the CAD) ──
    #   γ measured from +z around the hull x-axis:
    #   fin 1 = top (the dorsal fin in the front view), fins 2,3 = lower pair
    #   span direction f̂ = [0, sinγ, cosγ] ;  lift direction t̂ = x̂ × f̂
    γ_fins = [0.0, 2π/3, 4π/3]
    δ_fins = [δ1_s, δ2_s, δ3_s]
    fin_t(γ) = Num[0, cos(γ), -sin(γ)]
    fin_r(γ) = [x_fin, R_fin*sin(γ), R_fin*cos(γ)]

    fin_forces = [begin
        γ  = γ_fins[j]
        t̂  = fin_t(γ)
        rf = fin_r(γ)
        vt = dot(vbody(rf), t̂)             # local cross-flow along lift dir
        αj = δ_fins[j] - vt * inv_u        # deflection + weathervane term
        (Num.((qdyn * Afin * CLa * αj) .* t̂), rf, 1, :body)
    end for j in 1:3]
```

```
50
51      # ── force_inputs: (force_vector, point_local, body_id, frame) ──
52      force_inputs = vcat(Any[
53          ([0.0, 0.0, -W_val],  com_val,   1, :world),   # gravity at CG
54          ([0.0, 0.0,  Bb],     cb_val,    1, :world),   # buoyancy at CB (restoring pair)
55          ([T_thr_s, 0.0, 0.0], r_thr,     1, :body),    # thruster, along hull axis
56          ([Xu_*u_b + Xuu_*sabs(u_b), 0.0,
57            Zw_*w_b + Zww_*sabs(w_b)], [0,0,0], 1, :body),  # translational drag
58      ], fin_forces)
59
60      # ── torque_inputs: (torque_vector, body_id, frame) ──
61      torque_inputs = [
62          ([0.0, Mq_*qp_b + Mqq_*sabs(qp_b), 0.0], 1, :body),   # pitch damping
63          ([0.0, 0.0, Nr_*r_b + Nrr_*sabs(r_b)],   1, :body),   # yaw damping
64      ]
65  end
```

```
[[0.0, 0.0, -0.02]]
```

```
1   begin
2       # ═══════ REPLACED CELL — the parameter values (order = config_names) ═══════
3       # ⚠ Hydrodynamic coefficients are literature-plausible placeholders for a
4       #   small torpedo AUV — calibrate against CFD / tow-tank data when available.
5       inertia_mode  = :principal
6
7       config_names  = [:T_thr, :δ1, :δ2, :δ3,
8                        :ρw, :Bb, :veps,
9                        :Xu_, :Xuu_, :Zw_, :Zww_, :Mq_, :Mqq_, :Nr_, :Nrr_,
10                       :Xud, :Zwd, :Mqd, :Nrd,
11                       :Afin, :CLa]
12
13      config_values = [8.0, 0.0, 0.0, 0.0,          # thrust 8 N, fins centered
14                       1025.0, 139.0, 0.10,          # ρ, buoyancy (>W: floats up), v_eps
15                       -2.0, -9.0, -15.0, -40.0,     # surge/heave drag (lin, quad)
16                       -1.5, -4.0, -1.5, -4.0,       # pitch/yaw damping (lin, quad)
17                       -1.0, -12.0, -1.2, -1.2,      # added mass Xu̇, Zẇ, Mq̇, Nṙ
18                       0.004, 2.8]                   # fin area, lift slope
19
20      mass_values    = [m_hull_val]
21      inertia_values = [[0.05, 1.0, 1.0]]           # slender body: Ixx ≪ Iyy = Izz
22      com_values     = [com_val]
23  end
```

$$\left[\frac{\mathrm{d}\,\mathbf{xh}(t)}{\mathrm{d}t}, \frac{\mathrm{d}\,\mathbf{zh}(t)}{\mathrm{d}t}, \frac{\mathrm{d}\,\theta\mathbf{h}(t)}{\mathrm{d}t}, \frac{\mathrm{d}\,\psi\mathbf{h}(t)}{\mathrm{d}t}\right]$$

```
# ═══════════ PROCESSING: build parameters, bodies, system ═══════════
begin
    B = length(chains)

    P, p_syms, param_inputs = make_params(collect(1:B);
        config = config_names, config_values = config_values,
        inertia_mode = inertia_mode)

    point_forces_built = [
        point_force(f[3], f[2], Num.(f[1]); frame = f[4])
        for f in force_inputs
    ]
    body_torques_built = [
        body_torque(τ[2], Num.(τ[1]); frame = τ[3])
        for τ in torque_inputs
    ]

    bodies = [make_body(i, chains[i];
                forces  = [f for f in point_forces_built if f.body_id == i],
                torques = [τ for τ in body_torques_built if τ.body_id == i])
              for i in 1:B]

    sys = multibody_system(bodies)

    inertial_vals = reduce(vcat,
        [vcat(mass_values[j], inertia_values[j], com_values[j]) for j in 1:B])
    p_vals = vcat(config_values, inertial_vals)
    p_value_dict = Dict(p_syms .=> p_vals)

    dq = TDv(q)
end
```

```
sympylify (generic function with 1 method)
```

```
begin
    # ═══════ statify / destatify / sympylify — the symbolic boundaries ═══════
    #
    #   statify   : ψ(t), D(ψ(t))  →  plain q_dummy[i]   (for build_function & SymPy)
    #   destatify : the exact inverse
    #   sympylify : statify → SymPy simplify → destatify  (clean KE, M, h)
    #
    subs(D) = x -> substitute(x, D; fold = Val(true))

    states_dyn = Num[q...; TD.(q)...]
    n_state = length(states_dyn)

    @variables q_dummy[1:n_state]
    states_dync = collect(q_dummy)

    statify_dict   = Dict(zip(states_dyn,  states_dync))
    destatify_dict = Dict(zip(states_dync, states_dyn))

    statify(expr)   = expr |> subs(statify_dict)
    destatify(expr) = expr |> subs(destatify_dict)

    sympylify(D;
        statify_dict = statify_dict,
        destatify_dict = destatify_dict
    ) = Num(
        D |>
        subs(statify_dict) |>
        sympy_simplify |>
        subs(destatify_dict)
    )
end
```

```
SymbolicModel((bodies = [(id = 1, chain = [   more], forces = [   more], torques = [   more])], po
```

```
begin
    # ═══════ REPLACED CELL — build the model, WITH added-mass kinetic energy ═══════
    #
    # ADDED MASS = kinetic energy of entrained fluid (Kirchhoff). Because it
    # enters T, compute_M_h automatically emits BOTH the added-mass block in
    # M(q) AND the added-mass Coriolis terms in h(q,q̇) — no extra machinery,
    # and the 45-class → 10-class switched ablation now covers fluid coupling.
    kin_auv = build_kinematics(sys, q, P)
    KE_auv  = compute_KE(sys, P, kin_auv)

    Xud = cfg(:Xud); Zwd = cfg(:Zwd); Mqd = cfg(:Mqd); Nrd = cfg(:Nrd)
    ωb_hull = R_hull' * Omega(T_hull)          # body-frame angular velocity

    T_fluid = (1//2)*(-Xud)*νo[1]^2 + (1//2)*(-Zwd)*νo[3]^2 +
              (1//2)*(-Mqd)*ωb_hull[2]^2 + (1//2)*(-Nrd)*ωb_hull[3]^2

    Q_auv = compute_Q_total(sys, q, kin_auv)
    M_auv, h_auv = compute_M_h(KE_auv.KE_total + T_fluid, q)

    model = SymbolicModel(sys, P, q, TD.(q), vcat(q, TD.(q)), p_syms,
                    kin_auv, KE_auv, Q_auv, M_auv, h_auv)
end
```

```
((4, 4), 4)
```

```
size(model.M), length(model.h)                                  # (4,4), 4
```

```
begin
    # ═══════ BASELINE RUN — thrust on, small pitch offset, slight +buoyancy ═══════
    #
    # u = [x, z, θ, ψ, ẋ, ż, θ̇, ψ̇]   (8 states)
    u0 = [0.0, 0.0, 0.05, 0.0,   1.0, 0.0, 0.0, 0.0]
    u_test = u0                          # keeps the M-check cell working

    sol = run_simulation(model, u0, (0.0, 30.0); p = p_vals,
                    solver = AutoTsit5(Rosenbrock23(autodiff = AutoFiniteDiff())),
                    reltol = 1e-6, abstol = 1e-6)
    println("retcode = ", sol.retcode, ", steps = ", length(sol.t))
end
```

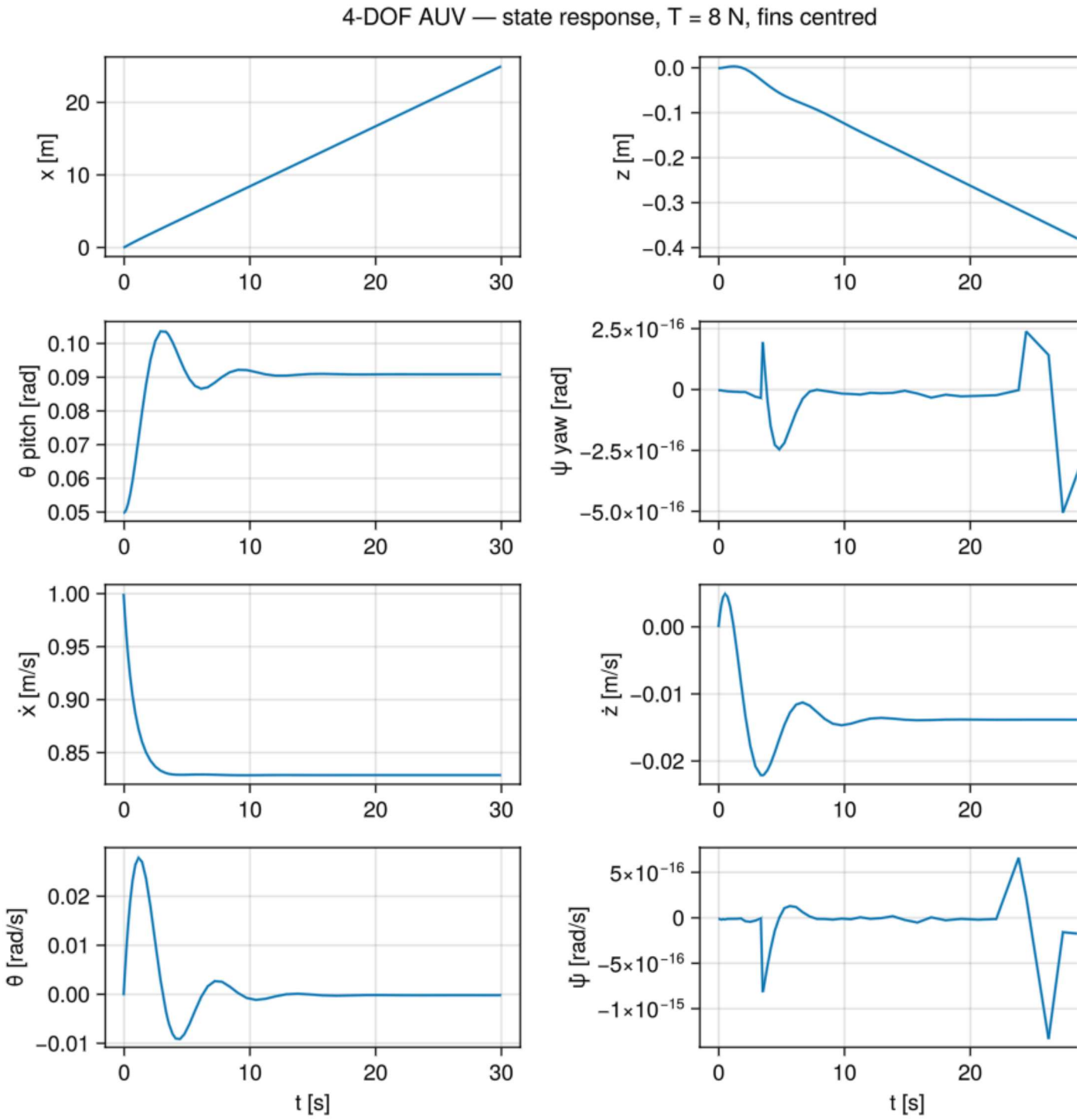
```
retcode = Success, steps = 49
```

```
begin
    # statified M, plain-symbol substitution with folding
    M_fun_chk = build_function(statify.(model.M), states_dync, model.p_syms;
                    ⇥ ⇥     expression = Val(false), cse = true)[1]
    M_stat = statify.(model.M)
    num_dict = Dict(vcat(states_dync .=> u_test, p_syms .=> p_vals))

    M_substituted = Symbolics.value.(subs(num_dict).(M_stat))

    err = maximum(abs.(Float64.(M_substituted) .- M_fun_chk(u_test, p_vals)))
    println("max |difference| = ", err)
    println("AGREE: ", err < 1e-12)
end
```

```
max |difference| = 0.0
AGREE: true
```




```
begin
    # ═══════ DELIVERABLE — all 8 states over time ═══════
    state_names = ["x [m]", "z [m]", "θ pitch [rad]", "ψ yaw [rad]",
                   "ẋ [m/s]", "ż [m/s]", "θ̇ [rad/s]", "ψ̇ [rad/s]"]
    figS = Figure(size = (750, 750))
    for i in 1:8
        r, c = fldmod1(i, 2)
        ax = Axis(figS[r, c], ylabel = state_names[i],
                  xlabel = r == 4 ? "t [s]" : "")
        lines!(ax, sol.t, [uu[i] for uu in sol.u])
    end
```

```
    Label(figS[0, :], "4-DOF AUV — state response, T = 8 N, fins centred",
          fontsize = 15, tellwidth = false)
    figS
end
```

```
begin
    uf = sol.u[end]
    println("t_end = ", sol.t[end])

    for (i, nm) in enumerate(["x", "z", "θ", "ψ"])
        println(
            rpad(nm, 4),
            round(uf[i], sigdigits = 4),
            "    d", nm, " = ",
            round(uf[i+4], sigdigits = 4)
        )
    end
end
```

```
t_end = 30.0
x    25.01    dx = 0.8289
z    -0.3997    dz = -0.01381
θ    0.09109    dθ = -1.537e-6
ψ    -2.561e-16    dψ = 1.257e-16
```

```
4×4 Matrix{Float64}:
  0.999  -0.0  -0.25  0.25
 -0.05    0.0  -5.0   5.0
  0.0     0.0  -2.1   2.1
  0.0    -2.4   1.24  1.24
```

```
begin
    # ═══════ B(q,q̇) — thruster/fin allocation matrix, EXTRACTED not written ═══════
    #
    # Q_total is linear in [T_thr, δ1, δ2, δ3] (thrust linear; fin lift ∝ CLα·δ),
    # so ∂Q/∂u_act is the exact allocation matrix — state-dependent through qdyn:
    # fin authority collapses quadratically as the vehicle slows.
    u_act = Num[T_thr_s, δ1_s, δ2_s, δ3_s]
    B_sym = jac(model.Q, u_act)

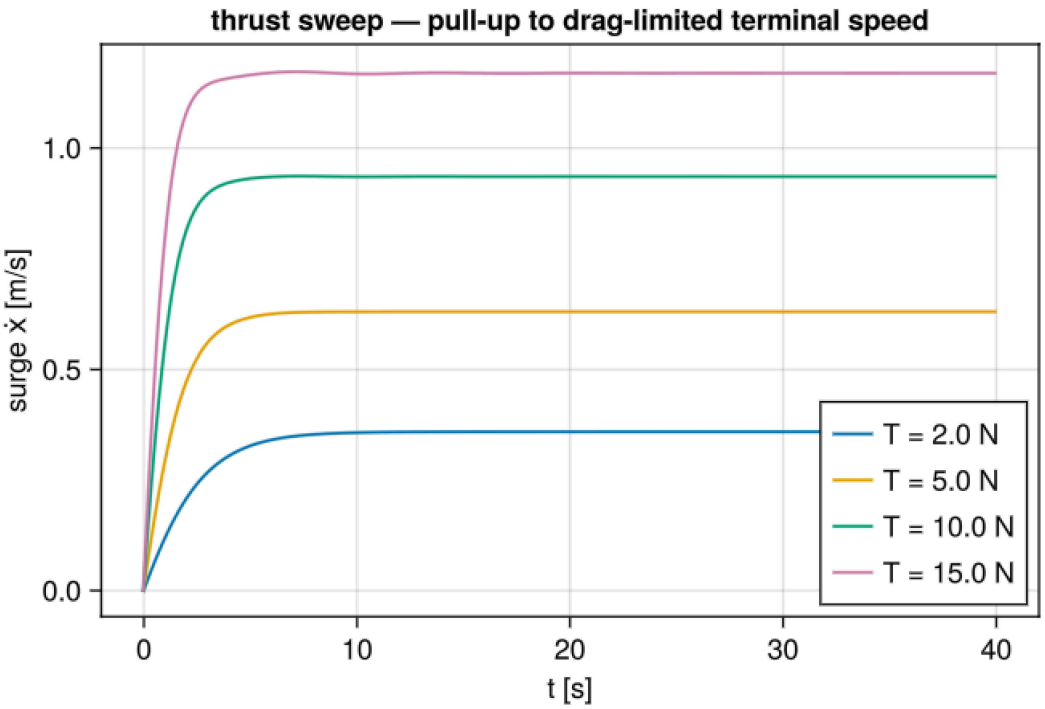
    B_fun = build_function(statify.(B_sym), states_dync, model.p_syms;
                    ↦    expression = Val(false), cse = true)[1]
    B_num = B_fun(u_test, p_vals)
    println("rank(B) at u0 = ", rank(B_num, rtol = 1e-8))
    round.(B_num, sigdigits = 3)
end
```

```
rank(B) at u0 = 3
```

```
(1, 2, 3, 4)
```

```
begin
    # ═══════ guarded RHS + parameter index lookup (replaces hardcoded p[15]/p[16])
    ═══════
    dyn_a! = compile_ode(model)
    function dyn!(du, u, p, tnow)
        # guard: |pitch| > 0.8 rad or non-finite → freeze (fin/hull model exits
        validity)
        if abs(u[3]) > 0.8 || !all(isfinite, u)
            du .= u .* 0
        else
            dyn_a!(du, u, p, tnow)
        end
    end

    pidx(s) = findfirst(x -> isequal(x, Symbolics.variable(s)), p_syms)
    idx_T  = pidx(:T_thr)
    idx_δ1 = pidx(:δ1)
    idx_δ2 = pidx(:δ2)
    idx_δ3 = pidx(:δ3)
    (idx_T, idx_δ1, idx_δ2, idx_δ3)
end
```

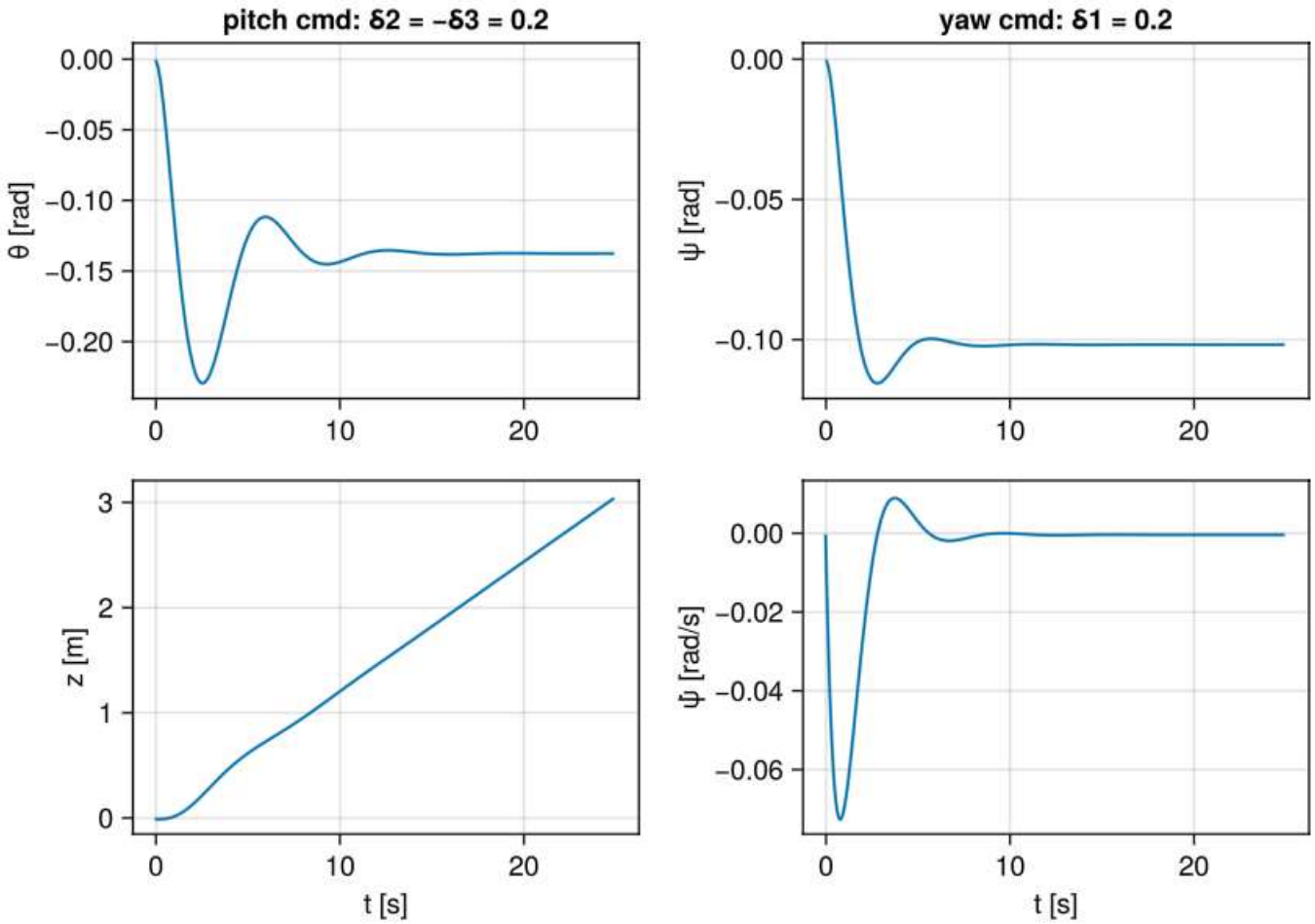



```
# ═══════ THRUST SWEEP: terminal surge speed set by quadratic drag (was V sweep) ═══════
begin
    thrusts = [2.0, 5.0, 10.0, 15.0]
    t_endT  = 40.0

    thr_sweep = Dict{Float64, Any}()
    for Tv in thrusts
        psp = copy(p_vals)
        psp[idx_T] = Tv
        s = solve(ODEProblem(dyn!, zeros(8), (0.0, t_endT), psp),
                  AutoTsit5(Rosenbrock23(autodiff = AutoFiniteDiff()));
                  reltol = 1e-6, abstol = 1e-6, saveat = 0.02, maxiters = 5_000_000)
        thr_sweep[Tv] = s
        println("T = ", rpad(Tv, 5), " retcode = ", s.retcode,
                "   u_end = ", round(s.u[end][5], digits = 3), " m/s")
    end

    fig_thr = Figure(size = (550, 380))
    ax_thr = Axis(fig_thr[1, 1], xlabel = "t [s]", ylabel = "surge ẋ [m/s]",
                  title = "thrust sweep — pull-up to drag-limited terminal speed")
    for Tv in thrusts
        lines!(ax_thr, thr_sweep[Tv].t, [u[5] for u in thr_sweep[Tv].u],
               label = "T = $(Tv) N")
    end
    axislegend(ax_thr, position = :rb)
    fig_thr
end
```

```
T = 2.0   retcode = Success   u_end = 0.362 m/s
T = 5.0   retcode = Success   u_end = 0.633 m/s
T = 10.0  retcode = Success   u_end = 0.939 m/s
T = 15.0  retcode = Success   u_end = 1.173 m/s
```




```
# ═══════ FIN STEP RESPONSES: 120° mixing → pitch and yaw channels (was kd study)
═══════
let
    # pitch command: differential lower fins (δ2 = -δ3) — y-components cancel,
    # z-components add → pure pitch/heave. yaw command: top fin δ1 alone.
    p_pitch = copy(p_vals); p_pitch[idx_T] = 8.0
    p_pitch[idx_δ2] =  0.20; p_pitch[idx_δ3] = -0.20

    p_yaw = copy(p_vals); p_yaw[idx_T] = 8.0
    p_yaw[idx_δ1] = 0.20

    u0f = zeros(8); u0f[5] = 1.0          # start at 1 m/s so fins have authority

    s_pitch = solve(ODEProblem(dyn!, u0f, (0.0, 25.0), p_pitch),
                    AutoTsit5(Rosenbrock23(autodiff = AutoFiniteDiff()));
                    reltol = 1e-6, abstol = 1e-6, saveat = 0.02, maxiters = 5_000_000)
    s_yaw   = solve(ODEProblem(dyn!, u0f, (0.0, 25.0), p_yaw),
                    AutoTsit5(Rosenbrock23(autodiff = AutoFiniteDiff()));
                    reltol = 1e-6, abstol = 1e-6, saveat = 0.02, maxiters = 5_000_000)

    fig_fin = Figure(size = (700, 500))
    ax1 = Axis(fig_fin[1, 1], ylabel = "θ [rad]", title = "pitch cmd: δ2 = -δ3 = 0.2")
    lines!(ax1, s_pitch.t, [u[3] for u in s_pitch.u])
    ax2 = Axis(fig_fin[2, 1], ylabel = "z [m]", xlabel = "t [s]")
    lines!(ax2, s_pitch.t, [u[2] for u in s_pitch.u])
    ax3 = Axis(fig_fin[1, 2], ylabel = "ψ [rad]", title = "yaw cmd: δ1 = 0.2")
```

```
    lines!(ax3, s_yaw.t, [u[4] for u in s_yaw.u])
    ax4 = Axis(fig_fin[2, 2], ylabel = "ψ̇ [rad/s]", xlabel = "t [s]")
    lines!(ax4, s_yaw.t, [u[8] for u in s_yaw.u])
    fig_fin
end
```

```
["dxh^2", "dxh*dzh", "dxh*dθh", "dxh*dψh", "dzh^2", "dzh*dθh", "dzh*dψh", "dθh^2", "dθh*dψh",
```

```
# ═══════ h CLASSIFICATION — numeric polarization, now eltype-generic ═══════
begin
    n_q = length(model.q)

    h_fun_cls = build_function(statify.(model.h), states_dync, model.p_syms;
                    ⇥ ⇥    expression = Val(false), cse = true)[1]

    clean(v) = replace(string(v), "(t)" => "")
    snames   = clean.(model.q)

    pair_list = [(i, j) for i in 1:n_q for j in i:n_q]
    labels    = [i == j ? "d$(snames[i])^2" : "d$(snames[i])*d$(snames[j])"
                 for (i, j) in pair_list]
    K = length(pair_list)

    function h_classes(u, p)
        T = promote_type(eltype(u), eltype(p))          # ← Float64 for sim, Num for
        proof
        qpos = u[1:n_q]; qdot = u[n_q+1:2n_q]
        ei = zeros(T, 2n_q); ei[1:n_q] .= qpos
        cdiag = Vector{Vector{T}}(undef, n_q)
        for i in 1:n_q
            ei[n_q+1:2n_q] .= 0; ei[n_q+i] = 1
            cdiag[i] = vec(h_fun_cls(ei, p))
        end
        parts = Vector{Vector{T}}(undef, K)
        for (k, (i, j)) in enumerate(pair_list)
            if i == j
                parts[k] = cdiag[i] .* qdot[i]^2
            else
                ei[n_q+1:2n_q] .= 0; ei[n_q+i] = 1; ei[n_q+j] = 1
                cij = vec(h_fun_cls(ei, p)) .- cdiag[i] .- cdiag[j]
                parts[k] = cij .* (qdot[i] * qdot[j])
            end
        end
        return parts
    end

    println("K = $K classes")                            # 4 DOF → K = 10
    labels
end
```

```
Attempting to print a symbolic expression in a Pluto notebook. Please run `import Latexify`
to enable pretty-printing of symbolic expressions. This warning will only display once.
```

```
K = 10 classes
```

u_sym = $[\mathbf{xh}(t), \mathbf{zh}(t), \theta\mathbf{h}(t), \psi\mathbf{h}(t), \frac{\mathrm{d}\,\mathbf{xh}(t)}{\mathrm{d}t}, \frac{\mathrm{d}\,\mathbf{zh}(t)}{\mathrm{d}t}, \frac{\mathrm{d}\,\theta\mathbf{h}(t)}{\mathrm{d}t}, \frac{\mathrm{d}\,\psi\mathbf{h}(t)}{\mathrm{d}t}]$

```
u_sym   = Num[model.q...; model.qdot...]
```

$$\left(\frac{\mathrm{d}\,\psi\mathbf{h}(t)}{\mathrm{d}t}\right)^2 \mathbf{c_y1\ m_1}\ \sin(\psi\mathbf{h}(t)) - \left(\frac{\mathrm{d}\,\psi\mathbf{h}(t)}{\mathrm{d}t}\right)^2 \mathbf{c_y1\ m_1}\ \sin(\psi\mathbf{h}(t)) - \cos^2(\theta\mathbf{h}(t))\ \mathbf{Xud}$$

```
# ═══════ SYMBOLIC PROOF: Σ h_classes == model.h ═══════
begin
    h_from_classes = reduce(+, h_classes(u_sym, p_syms))     # Σₖ parts_k, symbolic

    resid_cls = Symbolics.expand.(model.h[1] .- h_from_classes[1]) # polarization is
    exact
    println("Σ h_classes == model.h (expand) : ", all(iszero, resid_cls))
    resid_cls
end
```

```
Σ h_classes == model.h (expand) : false
```

```
run_switched (generic function with 1 method)
```

```
    # ═══════ SWITCHED DYNAMICS — switches in p, pitch guard (was roll guard) ═══════
    begin
        M_fun_sw = build_function(statify.(model.M), states_dync, model.p_syms;
                        ⇥ ⇥   expression = Val(false), cse = true)[1]
        Q_fun_sw = build_function(statify.(model.Q), states_dync, model.p_syms;
                        ⇥ ⇥   expression = Val(false), cse = true)[1]

        n_p = length(model.p_syms)

        function dynamics_sw!(du, u, p, tnow)
            @assert length(p) == n_p + K "p must be make_p(...) = [physical params; K
            switches]"
            # ── pitch guard: freeze past |θ| = 0.8 rad ──
            if abs(u[3]) > 0.8 || !all(isfinite, u)
                du .= u .* 0
                return nothing
            end
            pb = @view p[1:n_p]
            s  = @view p[n_p+1:n_p+K]
            parts = h_classes(u, pb)
            h_now = zeros(n_q)
            for k in 1:K
                s[k] != 0.0 && (h_now .+= s[k] .* parts[k])
            end
            rhs = vec(Q_fun_sw(u, pb)) .- h_now
            du[1:n_q]      .= u[n_q+1:2n_q]
            du[n_q+1:2n_q] .= M_fun_sw(u, pb) \ rhs
            return nothing
        end

        function make_p(switches::Dict = Dict(); base = p_vals)
            @assert length(base) == n_p "base params length ≠ n_p"
            s = ones(K)
            for (lab, v) in switches
                idx = findfirst(==(lab), labels)
                idx === nothing && error("unknown class '$lab' — check `labels`")
                s[idx] = float(v)
            end
            return vcat(base, s)
        end

        function show_switches(p)
            s   = p[n_p+1:end]
            off = [labels[k] for k in 1:K if s[k] == 0.0]
            sc  = [(labels[k], s[k]) for k in 1:K if !(s[k] in (0.0, 1.0))]
            println(K - length(off) - length(sc), " ON, ", length(off), " OFF",
                    isempty(off) ? "" : ("  → OFF: " * join(off, ", ")))
            isempty(sc) || println("scaled: ", sc)
        end

        # base kwarg lets you sweep T_thr/δ through idx_T/idx_δ without touching the model
        run_switched(u0s, tspan; switches = Dict(), base = p_vals,
                     solver = AutoTsit5(Rosenbrock23(autodiff = AutoFiniteDiff())),
```

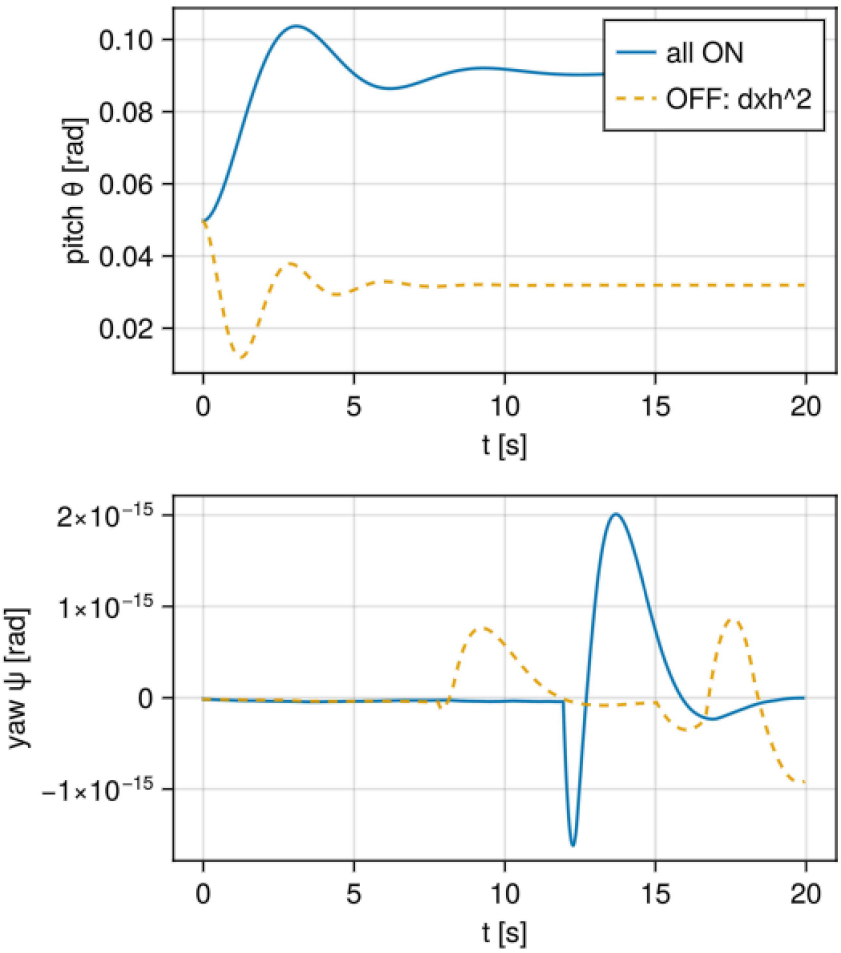
```
                    reltol = 1e-6, abstol = 1e-6) =
            solve(ODEProblem(dynamics_sw!, u0s, tspan, make_p(switches; base = base)),
                  solver; reltol, abstol, saveat = 0.01, maxiters = 5_000_000)
    end
```

```
(10, 10)
```

```
    K, length(labels)                                                    # 10, 10
```

```
class_table (generic function with 1 method)
```

```
    # ═══════ CLASS CONTRIBUTION TABLE ═══════
    begin
        function class_table(solx; p = p_vals, nsamp = 200)
            ts   = range(solx.t[1], solx.t[end]; length = nsamp)
            maxc = zeros(K); rmsc = zeros(K)
            for τ in ts
                parts = h_classes(solx(τ), p)
                for k in 1:K
                    v = norm(parts[k])
                    maxc[k]  = max(maxc[k], v)
                    rmsc[k] += v^2
                end
            end
            rmsc  = sqrt.(rmsc ./ nsamp)
            order = sortperm(maxc; rev = true)
            println(rpad("class", 18), rpad("max ‖h_k‖", 14), "rms ‖h_k‖")
            for k in order[1:min(15, K)]
                println(rpad(labels[k], 18),
                        rpad(string(round(maxc[k], sigdigits = 4)), 14),
                        round(rmsc[k], sigdigits = 4))
            end
            return (labels = labels[order], max = maxc[order], rms = rmsc[order])
        end
    end
```




```
# ═══════ SWITCH EXPERIMENT: baseline vs class-off (pitch/yaw axes) ═══════
let
    sol_on  = run_switched(u0, (0.0, 20.0))
    tbl     = class_table(sol_on)

    sol_off = run_switched(u0, (0.0, 20.0);
                  ⇥    switches = Dict(tbl.labels[1] => 0))

    fig_sw1 = Figure(size = (450, 500))
    ax1 = Axis(fig_sw1[1, 1], xlabel = "t [s]", ylabel = "pitch θ [rad]")
    lines!(ax1, sol_on.t,  [u[3] for u in sol_on.u],  label = "all ON")
    lines!(ax1, sol_off.t, [u[3] for u in sol_off.u], label = "OFF: " * tbl.labels[1],
           linestyle = :dash)
    axislegend(ax1)
    ax2 = Axis(fig_sw1[2, 1], xlabel = "t [s]", ylabel = "yaw ψ [rad]")
    lines!(ax2, sol_on.t,  [u[4] for u in sol_on.u])
    lines!(ax2, sol_off.t, [u[4] for u in sol_off.u], linestyle = :dash)
    fig_sw1
end
```

```
class              max ‖h_k‖     rms ‖h_k‖
dxh^2              0.7873        0.6845
dxh*d0h            0.2691        0.07736
dxh*dzh            0.1977        0.1259
dzh*d0h            0.002632      0.0007928
dzh^2              0.0005526     0.0002296
d0h^2              0.0002208     5.229e-5
```

```
dθh∗dψh          7.884e-19     9.843000000000001e-20
dxh∗dψh          3.462e-29     3.857e-30
dψh^2            2.59e-29      2.6919999999999996e-30
dzh∗dψh          2.355e-31     2.642e-32
```

```
# ═══════ SWITCH VERIFICATION ═══════
begin
    rng = MersenneTwister(1)

    h_fun_chk = build_function(statify.(model.h), states_dync, model.p_syms;
                    ⇥ ⇥    expression = Val(false), cse = true)[1]
    dyn_base! = compile_ode(model)          # baseline: rhs = Q − h (monolithic h)

    function rand_state()
        u = zeros(2n_q)
        u[3:4]  .= 0.1 .* randn(rng, 2)     # perturb pitch / yaw
        u[5]     = 0.5 + 1.5 * rand(rng)    # forward speed > 0
        u[6:8]  .= 0.3 .* randn(rng, 3)     # random velocities
        return u
    end

    println("── 1) h is homogeneous quadratic in q̇  (⇒ polarization is exact) ──")
    for trial in 1:3
        u = rand_state(); u2 = copy(u); u2[n_q+1:2n_q] .*= 2.0
        e = maximum(abs.(vec(h_fun_chk(u2, p_vals)) .- 4 .* vec(h_fun_chk(u, p_vals))))
        println("   |h(q,2q̇) − 4h(q,q̇)|∞ = ", e)
    end

    println("── 2) all switches ON: Σₖ parts[k] == h_fun exactly ──")
    for trial in 1:3
        u = rand_state()
        e = maximum(abs.(reduce(+, h_classes(u, p_vals)) .- vec(h_fun_chk(u, p_vals))))
        println("   |Σ parts − h|∞ = ", e)
    end

    println("── 3) make_p places s_k correctly: class OFF removes exactly parts[k] ──")
    let u = rand_state()
        parts  = h_classes(u, p_vals)
        k_test = argmax(norm.(parts))
        pk     = make_p(Dict(labels[k_test] => 0))
        show_switches(pk)
        s      = pk[n_p+1:end]
        h_sw   = reduce(+, [s[k] .* parts[k] for k in 1:K])
        h_ref  = reduce(+, parts) .- parts[k_test]
        println("   OFF '", labels[k_test], "': |h_switched − (h_full − part_k)|∞ = ",
                maximum(abs.(h_sw .- h_ref)))
    end

    println("── 4) full RHS: dynamics_sw!(all ON) == baseline compile_ode ──")
    du_a = zeros(2n_q); du_b = zeros(2n_q)
    for trial in 1:3
        u = rand_state()
        dyn_base!(du_a, u, p_vals, 0.0)
        dynamics_sw!(du_b, u, make_p(), 0.0)
        println("   |du_sw − du_base|∞ = ", maximum(abs.(du_a .- du_b)))
    end
end
```

```
── 1) h is homogeneous quadratic in q̇  (⇒ polarization is exact) ──
   |h(q,2q̇) - 4h(q,q̇)|∞ = 0.0
   |h(q,2q̇) - 4h(q,q̇)|∞ = 0.0
   |h(q,2q̇) - 4h(q,q̇)|∞ = 0.0
── 2) all switches ON: Σₖ parts[k] == h_fun exactly ──
   |Σ parts - h|∞ = 1.7763568394002505e-15
   |Σ parts - h|∞ = 1.5543122344752192e-15
   |Σ parts - h|∞ = 4.440892098500626e-16
── 3) make_p places s_k correctly: class OFF removes exactly parts[k] ──
9 ON, 1 OFF  → OFF: dxh*dzh
   OFF 'dxh*dzh': |h_switched - (h_full - part_k)|∞ = 8.881784197001252e-16
── 4) full RHS: dynamics_sw!(all ON) == baseline compile_ode ──
   |du_sw - du_base|∞ = 8.881784197001252e-16
   |du_sw - du_base|∞ = 5.551115123125783e-17
   |du_sw - du_base|∞ = 1.6653345369377348e-16
```

```
let
    checks = [
        ("K == 10 (4-DOF pairs)",    K == 10),
        ("h_classes is a function",  h_classes isa Function),
        ("make_p is a function",     make_p isa Function),
        ("labels has 10",            length(labels) == 10),
        ("make_p returns n_p+K",     length(make_p()) == n_p + K),
    ]
    for (name, ok) in checks
        println(rpad(name, 34), ok ? "PASS" : "*** FAIL ***")
    end
end
```

```
K == 10 (4-DOF pairs)              PASS
h_classes is a function            PASS
make_p is a function               PASS
labels has 10                      PASS
make_p returns n_p+K               PASS
```

```
Dict(:coriolis ⇒ ["dxh∗dzh", "dxh∗dθh", "dxh∗dψh", "dzh∗dθh", "dzh∗dψh", "dθh∗dψh"], :centri
```

```
# ═══════ PHYSICAL GROUPING of the 10 velocity-product classes ═══════
begin
    # No wheel spin DOFs on the AUV → :wheel_spin and :gyroscopic are empty;
    # every class is centrifugal (q̇ᵢ²) or Coriolis (q̇ᵢq̇ⱼ). The added-mass
    # Coriolis terms C_A(ν)ν live INSIDE these classes because T_fluid entered T.
    wheel_v = String[]
    att_v   = ["dθh", "dψh"]                # attitude rates

    function label_tokens(lab)
        occursin("^2", lab) && return (replace(lab, "^2" => ""),
                    ↹ ↹ ↹ ↹    replace(lab, "^2" => ""))
        p = String.(split(lab, "∗")); return (p[1], p[2])
    end

    function class_group(lab)
        a, b = label_tokens(lab)
        aw = a in wheel_v; bw = b in wheel_v
        if a == b
            return aw ? :wheel_spin : :centrifugal        # q̇ᵢ² terms
        elseif aw ⊻ bw                                    # exactly one spin rate
            return (aw ? b : a) in att_v ? :gyroscopic : :wheel_spin
        elseif aw && bw
            return :wheel_spin
        else
            return :coriolis                              # two distinct rates
        end
    end

    groups = Dict{Symbol,Vector{String}}()
    for lab in labels
        push!(get!(groups, class_group(lab), String[]), lab)
    end
    switch_off_group(g) = Dict(lab => 0 for lab in groups[g])

    for g in sort(collect(keys(groups)))
        println(rpad(string(g), 13), length(groups[g]), " classes")
    end
    groups
end
```

```
centrifugal  4 classes
coriolis     6 classes
```

```
Dict(:coriolis ⇒ 6, :centrifugal ⇒ 4)
```

```
Dict(g => length(v) for (g, v) in groups)                    # class counts per
group
```

```
# ═══════ SWITCH STUDY: baseline vs each physical group OFF ═══════
begin
    tspan_sw   = (0.0, 20.0)
    sol_base   = run_switched(u0, tspan_sw)                          # all ON
    group_sols = Dict(g => run_switched(u0, tspan_sw;
                     ⇥ ⇥ ⇥ ⇥ ⇥ switches = switch_off_group(g))
                       for g in keys(groups))
    println("done: ", join(string.(keys(group_sols)), ", "))
end
```

```
done: coriolis, centrifugal
```

```
# ═══════ DELIVERABLE TABLE ═══════
begin
    function group_contribution(sol, gidx)
        mx = 0.0; acc = 0.0
        for u in sol.u
            v = norm(reduce(+, h_classes(u, p_vals)[gidx]))
            isfinite(v) || continue
            mx = max(mx, v); acc += v^2
        end
        mx, sqrt(acc / length(sol.u))
    end

    function sol_metrics(sol)
        θ = [u[3] for u in sol.u]
        ok = sol.retcode == ReturnCode.Success && all(isfinite, θ)
        (; ok, maxθ = ok ? maximum(abs, θ) : Inf, endθ = θ[end])
    end

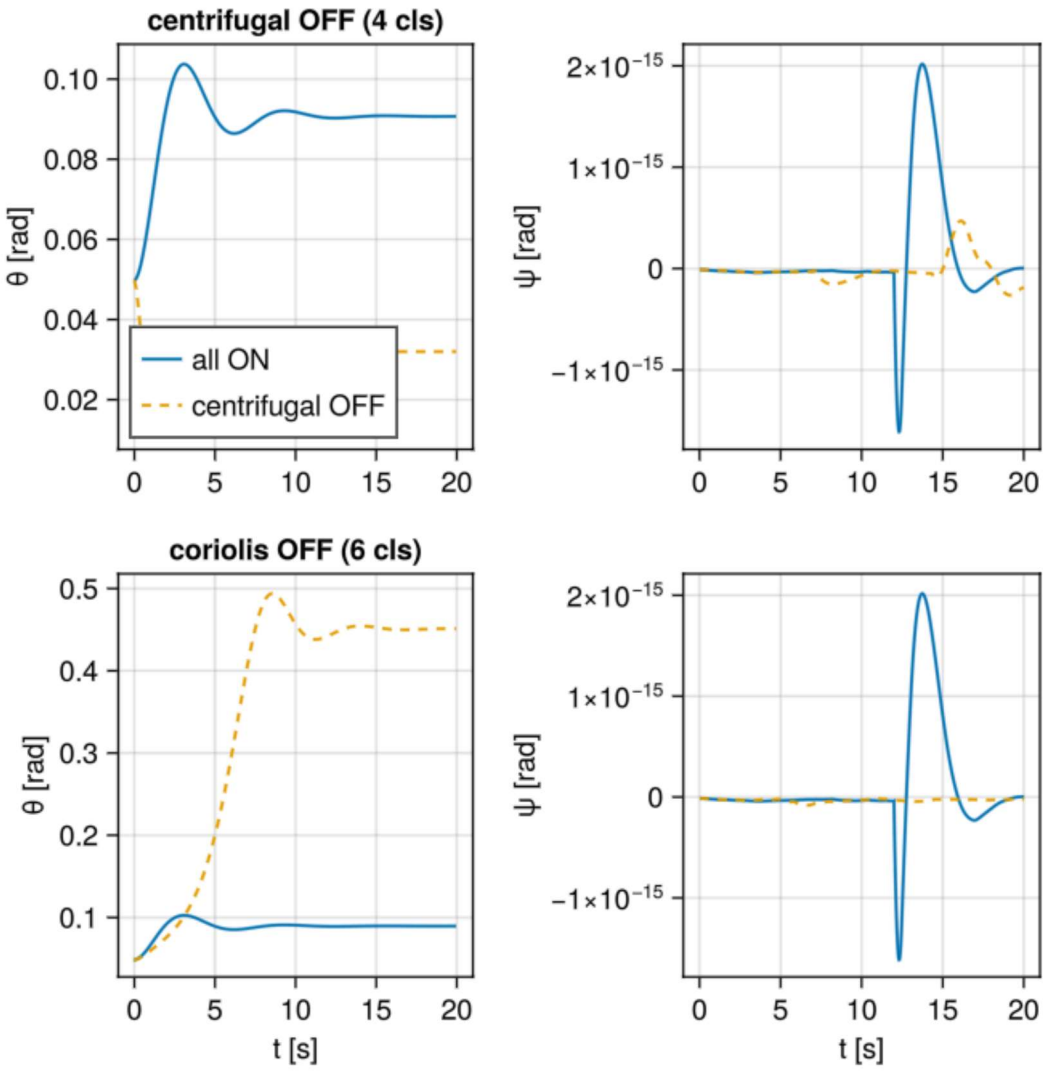
    verdict_of(m) = !m.ok        ? "BLEW UP"      :
                    m.maxθ > 0.8 ? "pitch guard"  :
                    m.maxθ > 0.3 ? "large"        : "stable"

    r3(x) = round(x, sigdigits = 3)
    println(rpad("group",13), rpad("#cls",6), rpad("rms‖h‖",12),
            rpad("max|θ|",10), rpad("θ(end)",10), "verdict")

    mb = sol_metrics(sol_base)
    println(rpad("(all ON)",13), rpad(K,6), rpad("–",12),
            rpad(r3(mb.maxθ),10), rpad(r3(mb.endθ),10), verdict_of(mb))

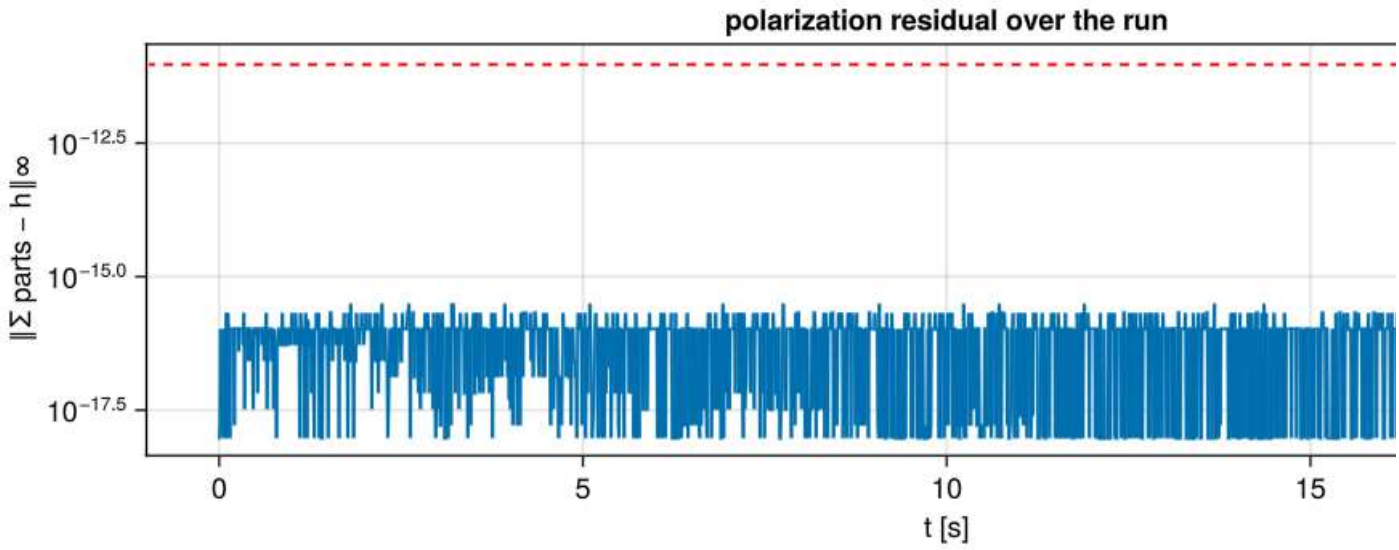
    for g in sort(collect(keys(groups)))
        gidx   = [findfirst(==(l), labels) for l in groups[g]]
        _, rms = group_contribution(sol_base, gidx)
        m      = sol_metrics(group_sols[g])
        println(rpad(string(g),13), rpad(length(groups[g]),6), rpad(r3(rms),12),
                rpad(r3(m.maxθ),10), rpad(r3(m.endθ),10), verdict_of(m))
    end
end
```

```
group        #cls  rms‖h‖      max|θ|    θ(end)    verdict
(all ON)     10    –           0.104     0.0911    stable
centrifugal  4     0.685       0.05      0.0323    stable
coriolis     6     0.148       0.496     0.453     large
```

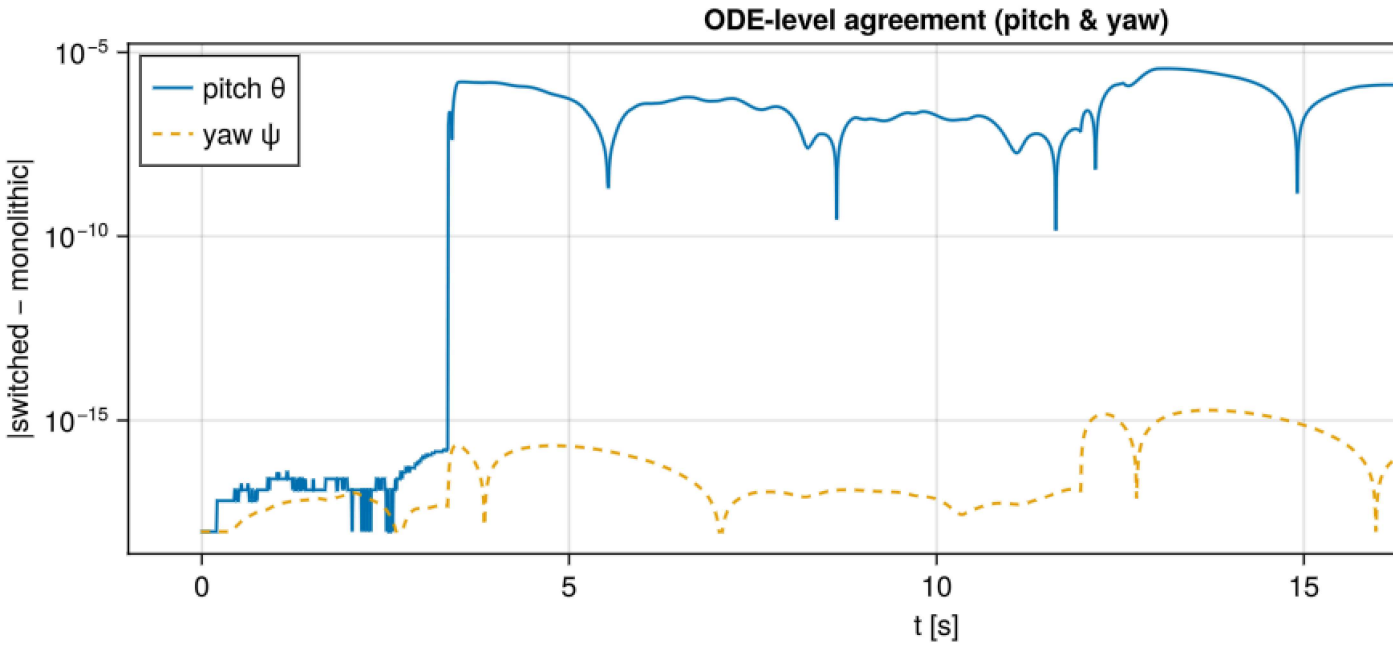



```
# ═══════ SWITCH-OFF PLOTS ═══════
begin
    glist  = sort(collect(keys(groups)))
    fig_sw = Figure(size = (550, 280 * length(glist)))
    for (r, g) in enumerate(glist)
        s    = group_sols[g]
        last = r == length(glist)
        axθ = Axis(fig_sw[r, 1], ylabel = "θ [rad]",
                   title  = "$(g) OFF ($(length(groups[g])) cls)",
                   xlabel = last ? "t [s]" : "")
        lines!(axθ, sol_base.t, [u[3] for u in sol_base.u], label = "all ON")
        lines!(axθ, s.t, [u[3] for u in s.u], linestyle = :dash, label = "$(g) OFF")
        r == 1 && axislegend(axθ, position = :lb)

        axψ = Axis(fig_sw[r, 2], ylabel = "ψ [rad]",
                   xlabel = last ? "t [s]" : "")
        lines!(axψ, sol_base.t, [u[4] for u in sol_base.u])
        lines!(axψ, s.t, [u[4] for u in s.u], linestyle = :dash)
    end
    fig_sw
end
```




```
# ═══════ residual ‖Σ h_classes − h_fun‖∞ along the trajectory ═══════
begin
    resid_t = [maximum(abs.(reduce(+, h_classes(u, p_vals)) .- vec(h_fun_cls(u,
    p_vals))))
               for u in sol_base.u]
    figR = Figure(size = (900, 300))
    axR  = Axis(figR[1, 1], xlabel = "t [s]", ylabel = "‖Σ parts − h‖∞",
                yscale = log10, title = "polarization residual over the run")
    lines!(axR, sol_base.t, max.(resid_t, 1e-18))
    hlines!(axR, [1e-11], color = :red, linestyle = :dash, label = "round-off ceiling")
    axislegend(axR, position = :rb)
    figR
end
```



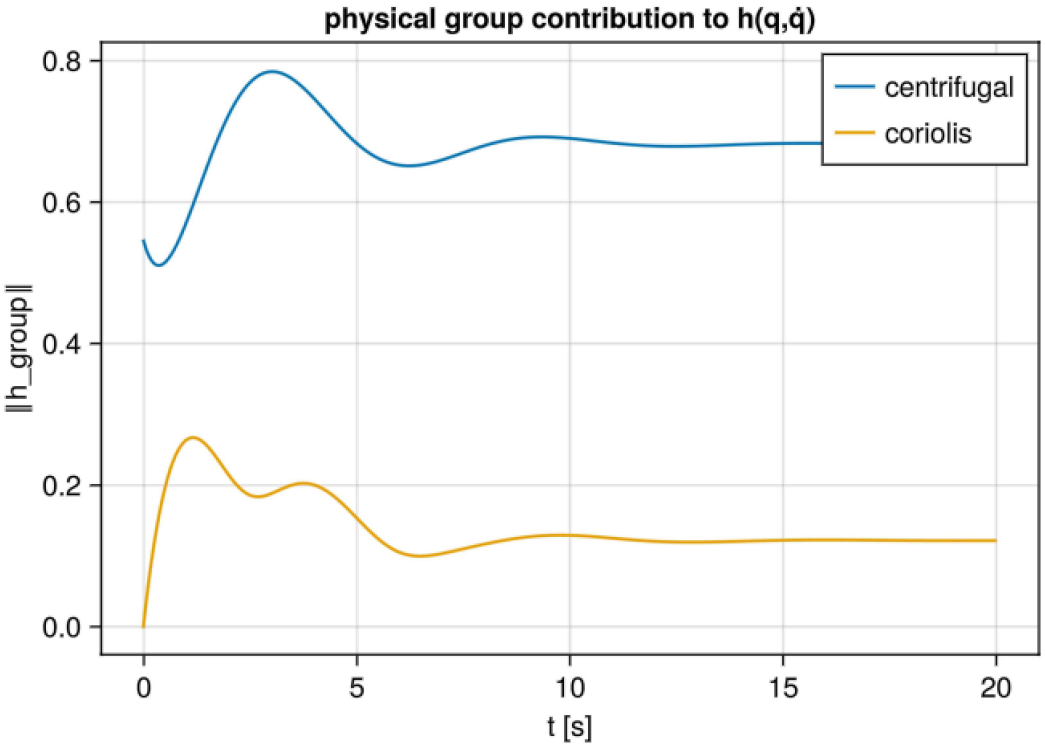

```
# ═══════ dynamics_sw!(all ON) reproduces compile_ode baseline ═══════
begin
    prob_mono = ODEProblem(compile_ode(model), u0, (0.0, 20.0), p_vals)
    sol_mono  = solve(prob_mono, AutoTsit5(Rosenbrock23(autodiff = AutoFiniteDiff()));
                      reltol = 1e-6, abstol = 1e-6, saveat = 0.01)   # same grid as
                    sol_base

    dθc = abs.([u[3] for u in sol_mono.u] .- [u[3] for u in sol_base.u])
    dψc = abs.([u[4] for u in sol_mono.u] .- [u[4] for u in sol_base.u])

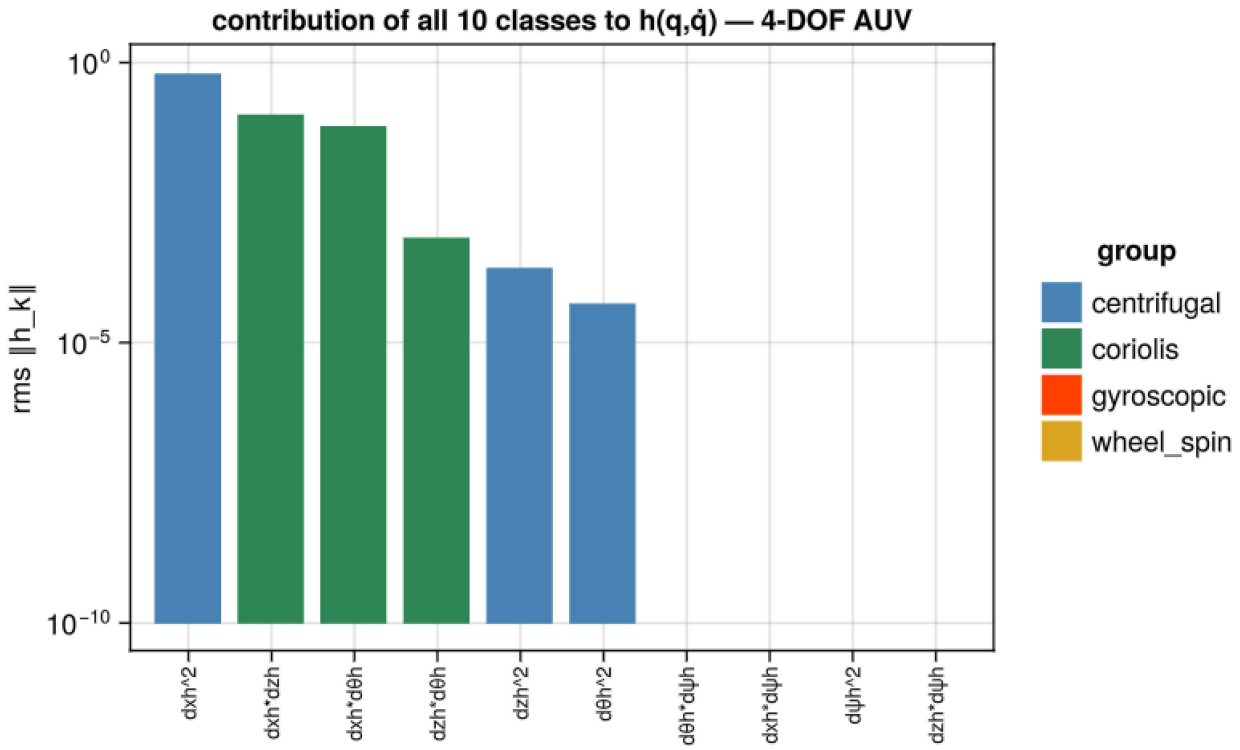
    figE = Figure(size = (900, 350))
    axE  = Axis(figE[1, 1], xlabel = "t [s]", ylabel = "|switched − monolithic|",
                yscale = log10, title = "ODE-level agreement (pitch & yaw)")
    lines!(axE, sol_mono.t, max.(dθc, 1e-18), label = "pitch θ")
    lines!(axE, sol_mono.t, max.(dψc, 1e-18), label = "yaw ψ", linestyle = :dash)
    axislegend(axE, position = :lt)
    figE
end
```

```
(ReturnCode.Success = 1, ReturnCode.Success = 1)
```

```
sol_base.retcode, sol_mono.retcode                                  # both Success
```




```
# ═══════ per-group contribution to ‖h‖ along the trajectory ═══════
begin
    gnames = sort(collect(keys(groups)))
    gidx   = Dict(g => [findfirst(==(l), labels) for l in groups[g]] for g in gnames)
    figG   = Figure(size = (550, 400))
    axG    = Axis(figG[1, 1], xlabel = "t [s]", ylabel = "‖h_group‖",
                  title = "physical group contribution to h(q,q̇)")
    for g in gnames
        c = [norm(reduce(+, h_classes(u, p_vals)[gidx[g]])) for u in sol_base.u]
        lines!(axG, sol_base.t, c, label = string(g))
    end
    axislegend(axG, position = :rt)
    figG
end
```




```
# ═══════ BAR CHART — all 10 classes, rms contribution to h(q,q̇) ═══════
let
    sol_bar = sol_base

    # rms over the run of ‖h_k(q)·q̇ᵢq̇ⱼ‖ for each class
    rms_k = zeros(K)
    nkeep = 0
    for u in sol_bar.u
        all(isfinite, u) || continue
        parts = h_classes(u, p_vals)
        nkeep += 1
        for k in 1:K
            v = norm(parts[k])
            isfinite(v) && (rms_k[k] += v^2)
        end
    end
    rms_k .= sqrt.(rms_k ./ max(nkeep, 1))

    # colour each bar by its physical group
    gname = ["centrifugal", "coriolis", "gyroscopic", "wheel_spin"]
    gcol  = [:steelblue, :seagreen, :orangered, :goldenrod]
    d     = Dict(:centrifugal => 1, :coriolis => 2, :gyroscopic => 3, :wheel_spin => 4)
    gid   = [d[first(g for (g, ls) in groups if lab in ls)] for lab in labels]

    ord = sortperm(rms_k; rev = true)
    println("top 3: ", labels[ord][1:3], "  values: ", round.(rms_k[ord][1:3],
    sigdigits=4))

    fig = Figure(size = (650, 400))
    ax  = Axis(fig[1, 1],
               title  = "contribution of all $(K) classes to h(q,q̇) — 4-DOF AUV",
               ylabel = "rms ‖h_k‖", yscale = log10,
```

```
                xticks = (1:K, labels[ord]),
                xticklabelrotation = π/2, xticklabelsize = 10)
        barplot!(ax, 1:K, max.(rms_k[ord], 1e-10), color = gcol[gid[ord]], fillto = 1e-10)
        xlims!(ax, 0.3, K + 0.7)
        Legend(fig[1, 2], [PolyElement(color = c) for c in gcol], gname, "group",
               framevisible = false)
        fig
    end
```

```
top 3: ["dxh^2", "dxh*dzh", "dxh*d0h"]  values: [0.6848, 0.126, 0.07753]
```

```
true
```

```
    length(p_syms) == length(p_vals)          # true — config and inertial in lockstep
```